Contents

1 RLHF & InstructGPT **1**
1.1 sft . . . 2
1.2 rm . . . 3
1.3 rl . . . 3
1.3.1 rl 流程概述 . . . 4
1.3.2 几个重要的 loss . . . 6
1.3.2.1 actor & actor loss . . . 6
1.3.2.2 critic & critic loss . . . 7
1.3.2.3 KL Penalty . . . 7
1.3.2.4 GAE . . . 8
1.3.2.5 entropy loss . . . 8
1.3.2.6 Policy kl . . . 9
1.3.3 两个采样 . . . 9
1.3.3.1 Old Policy Sampling（无 bp） . . . 9
1.3.3.2 New Policy Sampling（有 bp） . . . 9
1.3.4 开源 rlhf 库 . . . 10
1.3.4.1 openai 的 lm-human-preferences(gpt2 的 finetune) . . . 10
1.3.4.2 huggingface 的 TRL . . . 10
1.3.4.3 CarperAI 的 trlx . . . 10
1.3.4.4 allenai 的 RL4LMs . . . 10
1.3.4.5 RLHF workflow . . . 10
1.3.4.6 openrlhf . . . 10

2 Alignment：RLHF 变种 **10**
2.1 Alignment 综述 . . . 10
2.2 DPO . . . 11
2.3 SimPO . . . 12
2.4 TDPO . . . 12
2.5 CriticGPT . . . 12
2.6 DeRa . . . 12
2.7 Step-DPO . . . 12
2.8 RBR . . . 13
2.9 RLLR . . . 13
2.9.1 原始 RLHF 直接用在 NLU . . . 13
2.9.2 RLLR . . . 13
2.10 AMP . . . 13
2.11 Self-Taught Evaluators . . . 14
2.12 U-SOPHISTRY . . . 14
2.13 UNA . . . 14
2.14 Align-anything . . . 14
2.15 Beyond Preferences in AI Alignment . . . 14
2.16 TDPO-R . . . 14
2.17 self-critiquing . . . 14
2.18 Reward Centering . . . 14
2.19 Bradley-Terry models . . . 14
2.20 Weak-to-Strong Search . . . 15
2.21 reward hacking . . . 15
2.22 强化微调 . . . 15

1 RLHF & InstructGPT

OpenAI 魔改大模型，参数减少 100 倍！13 亿参数 InstructGPT 碾压 GPT-3

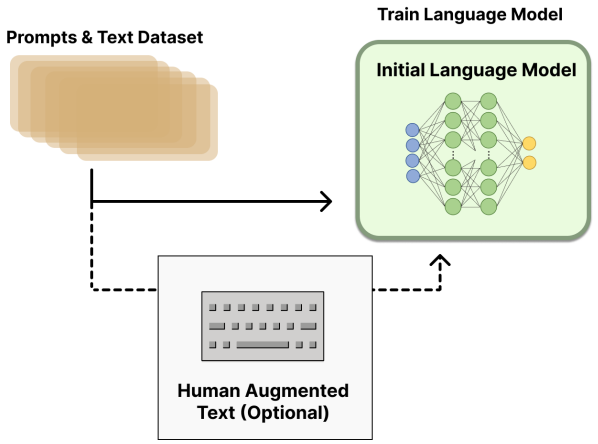
https://openai.com/blog/deep-reinforcement-learning-from-human-preferences/

Training language models to follow instructions with human feedback

https://huggingface.co/blog/zh/rlhf

- 预训练一个语言模型 (LM)；
- 聚合问答数据并训练一个奖励模型 (Reward Model，RM)，也叫偏好模型；
- 用强化学习 (RL) 方式微调 LM。

1.1 sft



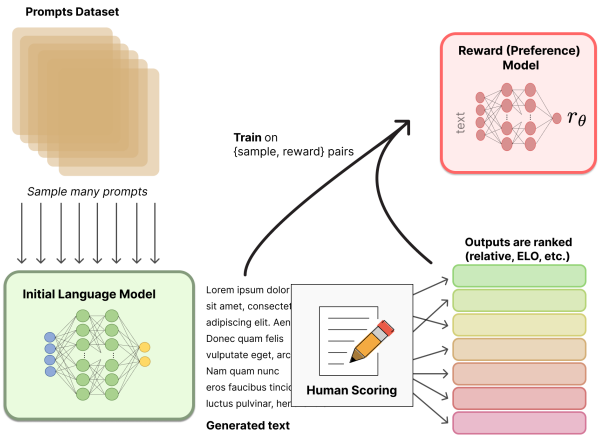
确保任务多样性的情况下，由标注人员编写 prompt 和一些生成式任务的期望输出。

- openai：instructGPT 使用小版本的 GPT-3，并对“更可取”（preferable）的人工生成文本微调
- Anthropic：1000w-520 亿参数的 transformer，并按“有用、诚实和无害”的标准在上下文线索上蒸馏原始 LM
- DeepMind：在Teaching language models to support answers with verified quotes提出的 GopherCite 模型中，用的是 2800 亿的模型 Gopher(Scaling language models: Methods, analysis & insights from training gopher)

剖析大模型 Pretrain 和 SFT 阶段的 Loss 差异

不管是 PreTraining 阶段还是 SFT 阶段，loss 函数都是一样的，只是计算的方式存在差异，PreTraining 阶段计算的是整段输入文本的 loss，而 SFT 阶段计算的是 **response 部分的 loss**。

1.2 rm



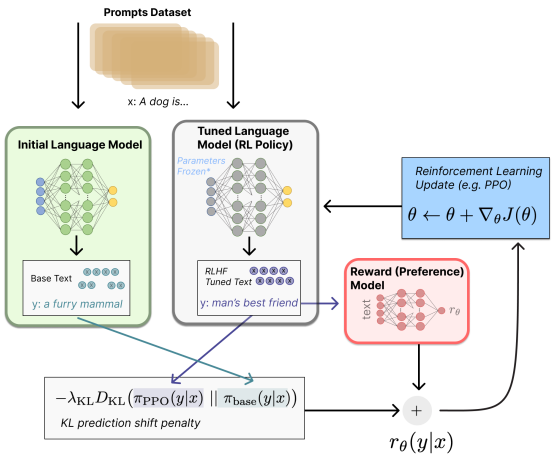
接收一系列文本并返回一个**标量奖励**，数值上对应人的偏好。我们可以用端到端的方式用 LM 建模，或者用模块化的系统建模 (比如**对输出进行排名**，再**将排名转换为奖励**) 。

- **模型选择**：RM 可以是另一个经过微调的 LM，也可以是根据偏好数据从头开始训练的 LM。Anthropic 提出了一种特殊的预训练方式，即用**偏好模型预训练** (Preference Model Pretraining，PMP) 来替换一般预训练后的微调过程，PMP **对样本的利用率更高**。
- **训练文本**：RM 的提示 - 生成对文本是从预定义数据集中采样生成的，并用初始的 LM 给这些提示生成文本。Anthropic 的数据主要是通过 Amazon Mechanical Turk 上的聊天工具生成的，并在 Hub 上可用，而 OpenAI 使用了用户提交给 GPT API 的 prompt。
- **训练奖励数值**：人工对 LM 生成的回答进行**排名**。起初我们可能会认为应该直接对文本标注分数来训练 RM，但是由于标注者的价值观不同导致这些分数未经过校准并且充满噪音，通过排名可以**比较多个模型各自的输出**并构建更好的规范数据集，这些不同的排名结果将被**归一化**为用于训练的标量奖励值。

目前成功的 RLHF 使用了和**要对齐的 LM** 具有**不同大小**的 LM：

- OpenAI：175B 的 LM 和 6B 的 RM
- Anthropic：使用的 LM 和 RM 从 10B 到 52B 大小不等
- DeepMind：使用了 70B 的 Chinchilla 模型分别作为 LM 和 RM

1.3 rl



直接微调整个 10B～100B+ 参数的成本过高，参考低秩自适应LoRA和 DeepMind 的Sparrow LM。目前多个组织找到的可行方案是使用策略梯度强化学习 (Policy Gradient RL) 算法、近端策略优化 (Proximal Policy Optimization，PPO) **微调初始 LM 的部分或全部参数**。

- 策略 (policy)：一个接受提示并返回一系列文本 (或文本的概率分布) 的 LM
- 行动空间（action space)：LM 的词表对应的所有词元 (一般在 50k 数量级)
- 观察空间 (observation space)：是可能的输入词元序列，也比较大 (词汇量 ^ 输入标记的数量)
- 奖励函数：偏好模型和策略转变约束 (Policy shift constraint) 的结合。

ppo 确定的奖励函数如下：

- 提示 x 输入初始 LM 和当前微调的 LM，分别得到输出文本 y_1 和 y_2
- 将来自当前策略的文本传给 RM 得到标量奖励 r_θ
- 将两个模型的生成文本进行比较计算差异的惩罚项，一般是输出词分布间的 KL 散度的缩放，即 $r = r_\theta - \lambda r_{KL}$，

惩罚项的好处：+ 用于惩罚策略在每个训练 batch 中生成大幅偏离初始模型，以确保模型输出合理连贯的文本。+ 如果没有这一项，可能导致模型在优化中生成乱码文本，以愚弄奖励模型提供高奖励值。

根据 PPO，按当前 batch 的奖励进行优化。PPO 是置信域优化（TRO，Trust Region Optimization）算法，用梯度约束确保更新步骤不会破坏学习过程的稳定性。

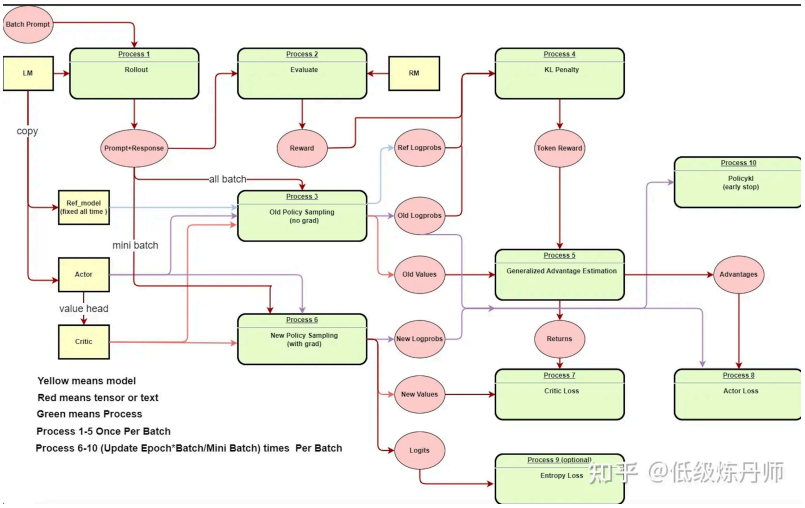
DeepMind 对 Gopher 用了类似的奖励设置，但用的是 A2C 来优化梯度。

1.3.1 rl 流程概述

https://zhuanlan.zhihu.com/p/635757674

Fine-Tuning Language Models from Human Preferences

Secrets of RLHF in Large Language Models Part I: PPO



- Rollout and Evaluation：从 prompt 库里抽样，使用语言模型生成 response，然后使用奖励模型（Reward Model, RM）给出奖励得分。这个得分反映了生成的 response 的质量，比如它是否符合人类的偏好，是否符合任务的要求等。
- Make experience：收集了一系列的“经验”，即模型的行为和对应的奖励。这些经验包括了模型生成的 response 以及对应的奖励得分。这些经验将被用于下一步的优化过程。
- Optimization：使用收集到的经验来更新模型的参数。具体来说，我们使用 PPO 算法来调整模型的参数，使得模型生成的 response 的奖励得分能够增加。PPO 算法的一个关键特性是它尝试保持模型的行为不会发生太大的改变，这有助于保证模型的稳定性。

官方代码 example

```
from tqdm import tqdm

for epoch, batch in tqdm(enumerate(ppo_trainer.dataloader)):
    query_tensors = batch["input_ids"]

    #### Get response from SFTModel
    response_tensors = ppo_trainer.generate(query_tensors, **generation_kwargs)
```

```
    batch["response"] = [tokenizer.decode(r.squeeze()) for r in response_tensors]

    #### Compute reward score
    texts = [q + r for q, r in zip(batch["query"], batch["response"])]
    pipe_outputs = reward_model(texts)
    rewards = [torch.tensor(output[1]["score"]) for output in pipe_outputs]

    #### Run PPO step
    stats = ppo_trainer.step(query_tensors, response_tensors, rewards)
    ppo_trainer.log_stats(stats, batch, rewards)

#### Save model
ppo_trainer.save_model("my_ppo_model")
```

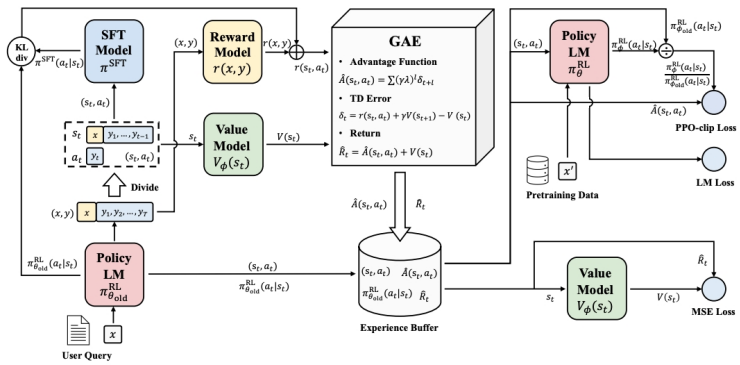


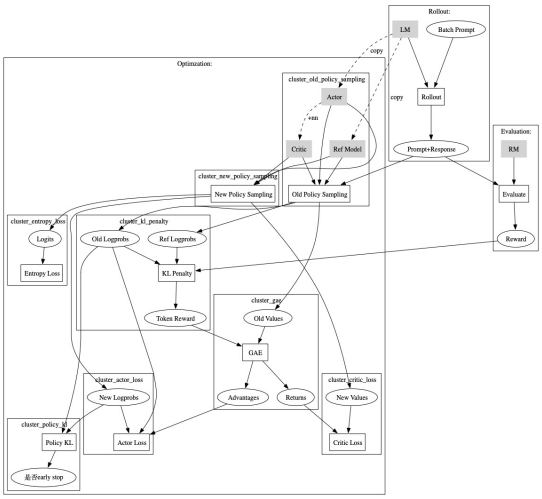
Figure 1: PPO workflow, depicting the sequential steps in the algorithm's execution. The process begins with sampling from the environment, followed by the application of GAE for improved advantage approximation. The diagram then illustrates the computation of various loss functions employed in PPO, signifying the iterative nature of the learning process and the policy updates derived from these losses.

- Rollout：根据策略（LM）生成轨迹（文本）。
  - 输入：Batch Prompt、LM
  - 输出：Prompt+Response
- Evaluate：对生成的轨迹进行评估（RM）。
  - 输入：Prompt+Response、RM
  - 输出：Reward
- Old Policy Sampling：计算并存储旧策略的概率、价值等值，
  - 输入：Ref_model、Actor、Critic、Prompt+Response
  - 输出：Ref Logprobs、Old Logprobs、Old Values
- KL Penalty：计算**当前策略**和**原始 LM** 之间的 KL 散度，用作对策略改变过快的惩罚项。
  - 输入：Ref Logprobs、Old Logprobs、Reward
  - 输出：Token Reward
- Generalized Advantage Estimation (GAE)：G。基于 old value(shape 是 (`batch_size`, `response_length`)) 和 reward 估计优势函数 A，它结合了所有可能的 n-step 进行 advantage 估计
  - 输入：Token Reward、Old Values
  - 输出：Advantages、Returns
- New Policy Sampling：
  - 输入 ref_model、actor、critic，从新的策略中采样概率等信息，
  - 输出 new logprobs、new values 和 logits，供 actor loss、critic loss 以及 entropy loss 用。
- Critic Loss：Critic 的目标是估计状态的价值函数，Critic loss 就是价值函数预测值和实际回报之间的差距。
  - 输入：New Values、Returns
  - 输出：critic 梯度更新
- Actor Loss：Actor 的目标是优化策略，Actor loss 就是基于优势函数的策略梯度。

  - 输入：Old Logprobs，New Logprobs、Advantages
  - 输出：actor 梯度更新
- Entropy Loss：为了增加探索性，通常会添加一个基于策略熵的正则项，它鼓励策略保持多样性。
  - 输入：Logits
  - 输出：entropy loss
- Policykl：这是对策略迭代过程的一个度量，它度量**新策略**和**旧策略**之间的差距。
  - 输入：Old Logprobs、New Logprobs
  - 输出：是否 early stop

在 PPO 中，策略优化的过程涉及到两个策略：一个是“旧的”策略，这是我们在开始每次优化迭代时使用的策略，另一个是“新的”策略，这是我们在优化过程中**不断更新**的策略。

自己整理重画的



1.3.2 几个重要的 loss

1.3.2.1 actor & actor loss

Actor 是**策略**，它决定文本会被怎么样生成，是从**策略网络**拷贝来的模拟整个智能体在环境中行动的网络。

优势函数表示在给定的状态下采取某个行动比遵循当前策略的期望回报要好多少。

Actor Loss 如下，用重要性采样比较在**旧策略**和**新策略**下行动的概率（Old Logprobs，New Logprobs），然后将这个比值（也就是 Importance Sampling 的权重）与**优势函数 Advantages** 相乘，得到了对 Actor Loss 的一个估计。

$$L = \pi_{new}/\pi_{old} * A$$

```
# 计算新旧策略下概率的比值
ratio = torch.exp(logprobs - old_logprobs)

# 计算未截断的策略梯度损失
pg_losses = -advantages * ratio

# 计算截断的策略梯度损失
pg_losses2 = -advantages * torch.clamp(ratio, 1.0 - self.config.cliprange,
     1.0 + self.config.cliprange)

# 选择两者中较大的作为最终的策略梯度损失
```

```
pg_loss = masked_mean(torch.max(pg_losses, pg_losses2), mask)

# 计算因为截断导致策略梯度损失改变的比例
pg_clipfrac = masked_mean(torch.gt(pg_losses2, pg_losses).double(), mask)
```

1.3.2.2 critic & critic loss

critic 是专门用来预测 actor 轨迹**每一步价值**的网络，actor 上加几个线性层能够给每个 token 预测一个值。任务是估计状态的价值函数，也就是预测从当前状态开始，通过遵循某个策略，期望能得到的总回报。

Critic Loss 是最小化它的预测价值与实际回报之间的差距，常用 mse

通过最小化 Critic Loss，Critic 的预测能力会逐渐提升。因为 Critic 的预测结果会被用来**估计每个行动的优势（Advantage）**，这个优势值又会被用来计算策略的更新（Actor Loss）。

```
# 将价值函数的预测值裁剪到一个范围内
vpredclipped = clip_by_value(
            vpreds, values - self.config.cliprange_value, values + self.config.cliprange_value
        )

# 计算裁剪前和裁剪后的价值函数损失
vf_losses1 = (vpreds - returns) ** 2
vf_losses2 = (vpredclipped - returns) ** 2

# 最终的价值函数损失是裁剪前和裁剪后损失的最大值的平均值的一半
vf_loss = 0.5 * masked_mean(torch.max(vf_losses1, vf_losses2), mask)

# 计算裁剪操作实际发生的频率
vf_clipfrac = masked_mean(torch.gt(vf_losses2, vf_losses1).double(), mask)
```

1.3.2.3 KL Penalty

用于保证经过强化学习后的模型（新策略 actor）不会过于偏离原始预训练模型（ref model）。

```
# 初始化两个列表来分别存储奖励和非得分奖励
rewards, non_score_rewards = [], []

# 使用 zip 函数并行遍历输入的得分、对数概率、参考模型的对数概率以及 mask
for score, logprob, ref_logprob, mask in zip(scores, logprobs,
        ref_logprobs, masks):
    # 计算 KL 散度，即模型的对数概率与参考模型的对数概率之间的差值
    kl = logprob - ref_logprob

    # 计算非得分奖励，即 KL 散度乘以 KL 控制器值的负值
    non_score_reward = -self.kl_ctl.value * kl
    non_score_rewards.append(non_score_reward)

    # 复制非得分奖励为新的奖励
    reward = non_score_reward.clone()

    # 找到 mask 中最后一个非零元素的索引，这表示输入序列的实际长度
    last_non_masked_index = mask.nonzero()[-1]
```

```
    # 对于最后一个非 mask 部分的 token，其奖励是偏好模型的得分加上 KL 散度
    reward[last_non_masked_index] += score

    # 将计算的奖励添加到奖励列表中
    rewards.append(reward)

# 返回包含所有奖励的张量以及包含所有非得分奖励的张量
return torch.stack(rewards), torch.stack(non_score_rewards)
```

1.3.2.4 GAE

GAE 是一种多步优势估计方法。它通过引入一个权衡参数 λ，在**单步 TD 误差**和**多步 TD 误差**之间进行权衡，从而**减小估计的方差**，提高学习的稳定性。其中 σ_{t+l} 是时间步 $t+l$ 的 TD 误差。

$$A_t = \sum_{l=0}^{k-1} (\lambda\eta)^l \sigma_{t+l}$$

$$\sigma_{t+l} = r_{t+l+1} + \eta V(s_{t+l+1}) - V(s_{t+l})$$

```
# 从后往前遍历整个生成的序列
for t in reversed(range(gen_len)):
    # 计算下一个状态的价值，如果当前状态已经是最后一个状态，则下一个状态的价值为 0
    nextvalues = values[:, t + 1] if t < gen_len - 1 else 0.0

    # 计算 delta，它是奖励加上衰减后的下一个状态的价值，然后减去当前状态的价值
    delta = rewards[:, t] + self.config.gamma * nextvalues - values[:, t]

    # 使用 delta 更新 lastgaelam，这是 GAE 公式的一部分
    lastgaelam = delta + self.config.gamma * self.config.lam * lastgaelam

    # 将计算的优势值添加到优势值列表中
    advantages_reversed.append(lastgaelam)

# 将优势值列表反向并转换为张量
advantages = torch.stack(advantages_reversed[::-1]).transpose(0, 1)

# 计算回报值，它是优势值加上状态值
returns = advantages + values
```

1.3.2.5 entropy loss

一个策略的熵越大，意味着这个策略选择各个动作的概率更加“平均”。在 actor 的 loss 里加熵，使得策略的熵尽可能大，从而有更多机会探索可能带来更好奖励的文本轨迹。

```
entropy = -torch.sum(logits* torch.log(logits + 1e-9), dim=-1).mean()
```

新实现：

```
    pd = torch.nn.functional.softmax(logits, dim=-1)
    entropy = torch.logsumexp(logits, axis=-1) - torch.sum(pd * logits, axis=-1)
```

1.3.2.6 Policy kl

在 PPO 中，KL 散度被用作一种约束，以确保在优化过程中新策略不会偏离旧策略太远。这是为了防止过度优化，因为过度优化可能会导致策略性能的大幅下降。

我们希望在优化目标函数的同时，满足以下的 KL 散度约束：

$$KL[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)] \leq \delta$$

在代码中，每个 mini batch 都会进行 early stop 的判定，如果计算出的 KL 散度大于 δ，那么就会停止这一轮的优化，以保证新策略不会偏离旧策略太远。

```
# 计算旧策略和新策略之间的 KL 散度
policykl = masked_mean(old_logprobs - logprobs, mask)
# old_logprobs 是旧策略下行为的概率的对数，logprobs 是新策略下的对数概率
# masked_mean 函数计算差异（old_logprobs - logprobs）的平均值，
# 但只考虑 mask 中对应元素为 True 的元素

# 检查计算出的 KL 散度（policykl）是否大于目标 KL 散度（self.config.target_kl）的 1.5 倍
if policykl > 1.5 * self.config.target_kl:
    self.optimizer.zero_grad()
    # 如果实际的 KL 散度超过了目标的 1.5 倍，那么策略改变过多，这步的梯度也不更新了。
    early_stop = True
    # 并设置 early_stop 标志为 True，表示应提前停止优化，以防止策略从旧策略进一步偏离
```

1.3.3 两个采样

1.3.3.1 Old Policy Sampling（无 bp）

是 **make experience** 的过程，计算并**存储**旧策略的概率、价值等值，来为后面更新的过程服务。

- Old Logprobs：从 “旧的” 策略 [即在这个 batch 数据中初始的 LM（initial actor）] 中计算每个 token 在旧的策略下的概率 Old Logprobs。
- Old Values：旧策略中每个**时间步**（每个 token 的预测结果）的价值，这个值由 critic 网络进行预测，critic 网络就是需要这个值的原因是 advantage 的计算依赖于 Old Values。
- Ref Logprobs：最最原始的 LM 对于每个时间步的概率预测，一般就是**固定不变的 gpt3**，计算这个值的目的是限制 actor 的更新，防止其偏离原始 gpt3 太远，他的实现在下一个步骤中。

```
all_logprobs, _, values, masks = self.batched_forward_pass(self.model, queries,
    responses, model_inputs)
ref_logprobs, _, _, _ = self.batched_forward_pass(self.ref_model, queries,
    responses, model_inputs)
```

1.3.3.2 New Policy Sampling（有 bp）

在**新的策略**（更新后的 actor）下对轨迹（文本）计算概率的过程，计算 Actor Loss，即策略梯度的损失。

Old Logprobs 是一次性一个 batch 的数据计算的，这是因为在一个 batch 中旧策略都是不变的；而 New Logprobs 是一个 mini batch 计算一次，这是因为新策略每个 mini batch 变一次。

1.3.4 开源 rlhf 库

https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/

影响 PPO 算法性能的 10 个关键技巧（附 PPO 算法简洁 Pytorch 实现）

1.3.4.1 openai 的 lm-human-preferences(gpt2 的 finetune)

https://github.com/openai/lm-human-preferences

1.3.4.2 huggingface 的 TRL

https://github.com/huggingface/trl

1.3.4.3 CarperAI 的 trlx

https://github.com/CarperAI/trlx

1.3.4.4 allenai 的 RL4LMs

https://github.com/allenai/RL4LMs

1.3.4.5 RLHF workflow

仅靠开源数据复刻出 LLaMA3 指令学习效果，在线迭代 RLHF 全流程解决方案来了

Iterative Preference Learning from Human Feedback: Bridging Theory and Practice for RLHF under KL-Constraint

RLHF Workflow: From Reward Modeling to Online RLHF

对应代码:

- https://github.com/RLHFlow/RLHF-Reward-Modeling
- https://github.com/RLHFlow/Online-RLHF

1.3.4.6 openrlhf

这个团队做了 OpenAI 没 Open 的技术，开源 OpenRLHF 让对齐大模型超简单

OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework

https://github.com/OpenLLMAI/OpenRLHF

2 Alignment：RLHF 变种

2.1 Alignment 综述

大模型微调（八）：SFT for Alignment 总结纪要

2024 年大模型 Alignment 偏好优化技术：从 PPO, SPO 到 MCTS-DPO

https://alignmentsurvey.com/

中文版

AI Alignment: A Comprehensive Survey

2.2 DPO

Direct preference optimization: Your language model is secretly a reward model

https://github.com/eric-mitchell/direct-preference-optimization

在 contextual bandit 的设定下，DPO 通过数学推导，得到了**奖励函数与最优策略之间的直接映射**，消除了 RLHF 过程中的奖励建模阶段，其中 $Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right)$ 是 partition function，

$$r(x, y) = \beta \log \frac{\pi_r(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x)$$

代入 Bradley-Terry 模型，可以得到

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right]$$

其中：

- x：来自偏好数据集的 prompt
- y_w：来自偏好数据集的获胜 response
- y_l：来自偏好数据集的失败 response

其具体含义如下，其中 $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$：

$$\begin{aligned} &\nabla_\theta \mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = \\ &-\beta \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\underbrace{\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))}_{\text{higher weight when reward estimate is wrong}} [\underbrace{\nabla_\theta \log \pi(y_w \mid x)}_{\text{increase likelihood of } y_w} - \underbrace{\nabla_\theta \log \pi(y_l \mid x)}_{\text{decrease likelihood of } y_l}]] \end{aligned}$$

具体实现：

```python
def preference_loss(policy_chosen_logps: torch.FloatTensor,
                    policy_rejected_logps: torch.FloatTensor,
                    reference_chosen_logps: torch.FloatTensor,
                    reference_rejected_logps: torch.FloatTensor,
                    beta: float,
                    label_smoothing: float = 0.0,
                    ipo: bool = False,
                    reference_free: bool = False) -> Tuple[torch.FloatTensor,
                        torch.FloatTensor, torch.FloatTensor]:

    pi_logratios = policy_chosen_logps - policy_rejected_logps
    ref_logratios = reference_chosen_logps - reference_rejected_logps

    if reference_free:
        ref_logratios = 0

    logits = pi_logratios - ref_logratios  # also known as h_{\pi_\theta}^{y_w,y_l}

    if ipo:
        # Eq. 17 of https://arxiv.org/pdf/2310.12036v2.pdf
        losses = (logits - 1/(2 * beta)) ** 2
    else:
        # Eq. 3 https://ericmitchell.ai/cdpo.pdf;
        # label_smoothing=0 gives original DPO
```

```
        # (Eq. 7 of https://arxiv.org/pdf/2305.18290.pdf)
        losses = -F.logsigmoid(beta * logits) * (1 - label_smoothing) -
            F.logsigmoid(-beta * logits) * label_smoothing

    chosen_rewards = beta * (policy_chosen_logps - reference_chosen_logps).detach()
    rejected_rewards = beta * (policy_rejected_logps - reference_rejected_logps).detach()

    return losses, chosen_rewards, rejected_rewards
```

RLHF 与 DPO 对比：

- RLHF 优化的是 token-level 的 value functions，对应的是 terminate state 的 sparse reward。
- DPO 则是在 context bandit 的设定下，将整个 response 看成一个 arm。虽然 token 是一个个生成的，但在 RL 里一般 dense rewards 效果更好。

2.3 SimPO

全面超越 DPO：陈丹琦团队提出简单偏好优化 SimPO，还炼出最强 8B 开源模型

SimPO: Simple Preference Optimization with a Reference-Free Reward

https://github.com/princeton-nlp/SimPO

2.4 TDPO

从 RLHF 到 DPO 再到 TDPO，大模型对齐算法已经是「token-level」

Token-level Direct Preference Optimization

https://github.com/Vance0124/Token-level-Direct-Preference-Optimization

2.5 CriticGPT

GPT-4 批评 GPT-4 实现「自我提升」！OpenAI 前超级对齐团队又一力作被公开

LLM Critics Help Catch LLM Bugs

CriticGPT 依旧是自回归模型。标注者先向 ChatGPT 的响应输出中人为注入一些微妙的错误，CriticGPT 针对这些有错误的答案生成批评意见，之后再由人类训练师为批评意见进行打分排名。

2.6 DeRa

ICML 2024 Spotlight | 在解码中重新对齐，让语言模型更少幻觉、更符合人类偏好

Decoding-time Realignment of Language Models

https://github.com/liutianlin0121/decoding-time-realignment

2.7 Step-DPO

贾佳亚团队新作：10k 数据让大模型数学能力超 GPT-4

https://github.com/dvlab-research/Step-DPO

Step-DPO: Step-wise Preference Optimization for Long-chain Reasoning of LLMs

超越 DPO！大模型精细化对齐之 Step-DPO

2.8 RBR

RLHF 不够用了，OpenAI 设计出了新的奖励机制

Rule Based Rewards for Language Model Safety

https://openai.com/index/improving-model-safety-behavior-with-rule-based-rewards/

https://github.com/openai/safety-rbr-code-and-data

2.9 RLLR

ACL2024 | RLHF 在腾讯广告自然语言理解任务上的优化及应用

Enhancing Reinforcement Learning with Label-Sensitive Reward for Natural Language Understanding

更适合 NLU 任务的强化学习方法 RLLR（Reinforcement Learning with Label-Sensitive Reward），与 RLHF 相比可以一致地提升多种 NLU 任务上的标签准确率；进一步地，通过结合 RLHF 和 RLLR 的两个 Reward Model，RLLR-mixed 方法可以在标签准确率和理由质量上取得全面提升。

2.9.1 原始 RLHF 直接用在 NLU

- 将 NLU 任务改写为自然语言的形式，让模型对同一个问题输出多条回复，每条回复包括两部分：
  - 理由（rationale）：可以视为思维链的一种简化版本
  - 标签（label）
- 对同一个问题的不同回复进行排序，并处理成回复对（pair）的形式。
  - 标签敏感对（label-sensitive pair）：标签不同的回复对
  - 理由敏感对（rationale-sensitive pair）：**标签相同、理由不同**的回复对

在 7 个公开数据集上，理由敏感对的占比都超过 75%。在理由敏感对中，两个不同的理由会导向相同的标签，理论上说，模型在这些数据上进行训练只会使理由**更符合标注者的偏好**，但**对标签的准确性没有帮助**；而在 NLU 任务上，我们实际**更关心标签的准确性**，因此存在模型训练目标与评估指标不一致的问题。

2.9.2 RLLR

- SFT 训练：除了标签之外，我们还为数据集标注了理由，参考 CoT 的方式，先让模型生成一段理由，再根据理由输出标签，训练得到 Policy Model；
- Reward Model 训练：训练两个 reward model：
  - **标签敏感 RM**：解决原始 RLHF 中标签敏感对比例较低的问题：对于一条训练集中的样本，我们已有正确的标签标注，并且也知道所有可能的标签集合，因此可以**随机采样一个错误的标签**，并**为错误的标签标注理由**，与正确标签 + 理由构成**标签敏感对**，并基于此方法构造的数据训练单独的 Reward Model；
  - **理由敏感 RM**：对于训练集中的每条样本，我们基于正确的标签采样多条理由，根据**理由的生成质量**来构建理由敏感对，并训练理由敏感的 Reward Model，此处理由质量可以采用**人工判断或 AI 辅助**的方式标注，根据准确性、一致性、逻辑性、事实性、相关性和信息完整性进行排序。
- PPO 训练：使用 Reward Model 和的 Policy Model 进行强化学习训练。
  - RLLR：只用标签敏感 RM 训练
  - RLLR-mixed：用两个 RM 训练，最终 reward 如下，$r_{\phi 1}$ 是标签敏感 RM 的输出，$r_{\phi 2}$ 是理由敏感 RM 的输出，大部分情况下，$r_{\phi 2} < r_{\phi 1}$，为了强化各自的作用，当 $r_{\phi 1}$ 较小时，由 $r_{\phi 1}$ 主导，当 $r_{\phi 1} > \lambda$ 时，截断到 λ，由 $r_{\phi 2}$ 主导：

$$r_M(q,a) = \begin{cases} r_{\phi 1}(q,a) + r_{\phi 2}(q,a) & r_{\phi 1}(q,a) < \lambda \\ \lambda + r_{\phi 2}(q,a) & r_{\phi 1}(q,a) \geq \lambda \end{cases}$$

2.10 AMP

无需人工/GPT-4V 排序，针对多模态大模型的全自动多级偏好学习

Automated Multi-level Preference for MLLMs

https://github.com/takomc/amp

2.11 Self-Taught Evaluators

Self-Taught Evaluators

https://github.com/facebookresearch/RAM/tree/main/projects/self_taught_evaluator

https://huggingface.co/facebook/Self-taught-evaluator-llama3.1-70B

2.12 U-SOPHISTRY

AI 会「说谎」，RLHF 竟是帮凶

Language Models Learn to Mislead Humans via RLHF

2.13 UNA

综合 RLHF、DPO、KTO 优势，统一对齐框架 UNA 来了

2.14 Align-anything

全模态对齐框架 align-anything 来了：实现跨模态指令跟随

https://github.com/PKU-Alignment/align-anything

2.15 Beyond Preferences in AI Alignment

人类自身都对不齐，怎么对齐 AI？新研究全面审视偏好在 AI 对齐中的作用

Beyond Preferences in AI Alignment

2.16 TDPO-R

与 OpenAI o1 技术理念相似，TDPO-R 算法有效缓解奖励过优化问题

Confronting Reward Overoptimization for Diffusion Models: A Perspective of Inductive and Primacy Biases

https://github.com/ZiyiZhang27/tdpo

2.17 self-critiquing

Self-critiquing models for assisting human evaluators

有 AI 辅助的标注员能比无 AI 辅助的标注员找出更多的摘要中的错误

2.18 Reward Centering

强化学习之父 Richard Sutton 给出一个简单思路，大幅增强所有 RL 算法

Reward Centering

2.19 Bradley-Terry models

思考 Bradley-Terry 和 Reward Modeling 这一年

Rethinking the Bradley-Terry Models in Preference-based Reward Modeling: Foundation, Theory, and its Alternatives

2.20 Weak-to-Strong Search

NeurIPS 2024 | 小模型引导大模型生成，无需微调实现弱到强泛化！

Weak-to-Strong Search: Align Large Language Models via Searching over Small Language Models

https://github.com/ZHZisZZ/weak-to-strong-search

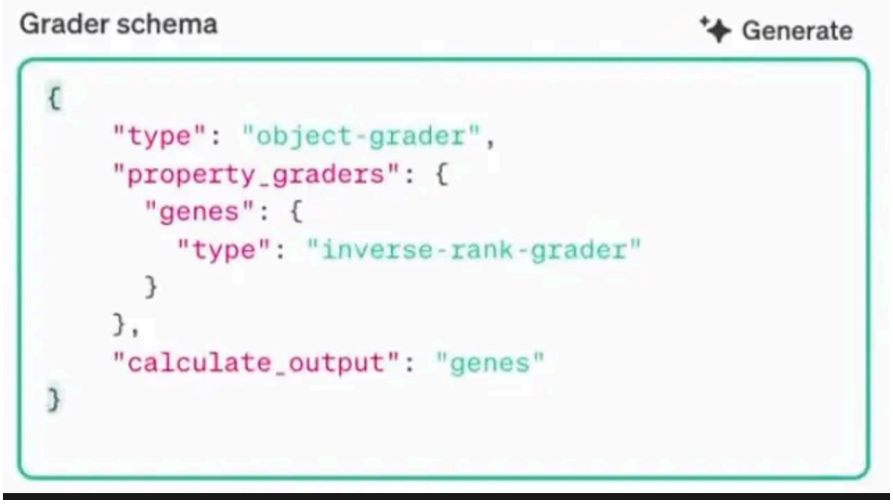
2.21 reward hacking

翁荔离职 OpenAI 后第一个动作：万字长文探讨 RLHF 的漏洞，网友们抢着传看

离职 OpenAI 后，翁荔博客首次上新，引众网友围观学习（中文全文）

Reward Hacking in Reinforcement Learning

2.22 强化微调

OpenAI 12 连发第 2 弹：强化微调，少量样本就能训练自己的专家模型

评分器（grader）会比较模型输出与正确答案，然后返回一个 0 到 1 之间的分数。0 表示模型的输出中不包含正确答案，而 1 表示正确答案在输出的第一个位置。例如正确答案在第 2 个位置，评分器可能会给出 0.7 的分数。